Superpod Run

What It Is and What It Produces

A Handoff Note

Joe Corneli | Hyperreal Enterprises | February 16, 2026

Context

Rob — this note explains the "superpod run," a batch processing job that takes the full StackExchange mathematics data dumps and transforms them into structured knowledge artifacts. The run is the next concrete step in the futon6 project, and its output feeds directly into preparations for First Proof Batch 2 (expected ~1 month from now).

The short version: we're taking 567K math.stackexchange threads and 100K MathOverflow threads, running them through an 11-stage pipeline (Stages 1–7 plus LWGM Stages 8–10), and producing typed artifacts for every thread — wiring diagrams, expression surfaces, hypergraphs, and a structural similarity index for nearest-neighbor retrieval by argument shape.

What Goes In

StackExchange publishes complete data dumps of every Q&A site. For mathematics we use two:

- **math.stackexchange.com —** 567K question threads. Undergraduate through graduate level. High volume, broad coverage.
- **mathoverflow.net —** 100K question threads. Research level. Narrower but deeper.

Each thread has a question, zero or more answers, and comments. The raw data is XML. Total: roughly 20 GB, compressing to about 5 GB.

What the Pipeline Does

The superpod job has eleven stages. Four are GPU-bound (2, 3, 6, 9b); the rest are streaming CPU passes. In the required superpod run we typically skip the LLM stages (3 and 6) and still produce the core deliverables.

#	Stage	What it does	HW
1	**Parse**	Stream XML into structured QA pairs	CPU
2	**Embed**	Dense vector embeddings of posts (similarity + clustering)	GPU
3	**Tag**	LLM tagging: 25 argument patterns per answer	GPU
4	**Cluster**	Topic clustering based on embeddings	CPU
5	**Terms + scopes**	Term dictionary + scope openers (Let/Assume/Define)	CPU
6	**Rev. morph.**	LLM situation reconstruction per QA pair	GPU
7	**Thread wiring**	Typed wiring: IATC + CT refs + ports	CPU
8	**Expr. surfaces**	Parse $...$ LaTeX into typed s-expressions	CPU
9a	**Hypergraphs**	Assemble typed hypergraph per thread	CPU
9b	**Graph embed**	R-GCN contrastive embedding (LWGM)	GPU
10	**Similarity idx**	FAISS k-NN index over structural embeddings	CPU

Run modes: core superpod run (skip LLM, run GPU where needed for embeddings and LWGM),

moist-run (generate prompt files for cloud LLM), and full GPU LLM runs (Stages 3 and 6). The critical output is Stage 7; Stages 8–10 extend it with expression-level structure and a global structural retrieval index.

What Comes Out

For every thread, Stage 7 produces a JSON wiring diagram with three levels:

1. **Thread level (discourse).** Each post becomes a node. Edges are typed by illocutionary act — assert, challenge, clarify, reference, retract, reform, etc. These capture the argumentative moves in the thread.
2. **Categorical level.** Each node is checked against a reference dictionary of 8 category-theory pattern types (adjunction, equivalence, fibration, etc.), extracted from 20K nLab wiki pages. IDF weighting avoids false positives.
3. **Port level.** Mathematical content is decomposed into input ports (assumptions, let-bindings) and output ports (conclusions, constraints). Edges carry port matches showing which outputs connect to which inputs, with scores boosted by CT reference weights.

So where a raw SE thread looks like "question → answer → comment," the wiring diagram looks more like: "question introduces terms X, Y via let-bindings; answer asserts conclusion Z via Cauchy-Schwarz referencing X; comment challenges the bound on Y with an adversative discourse marker."

Scale

	math.SE	MathOverflow
Threads	~567K	~100K
Nodes (posts)	~3M	~500K
Edges (typed)	~2.5M	~400K
Output size (est.)	~45 GB	~8 GB

The 200-thread pilot we ran this week produced 1,641 nodes, 1,441 edges, and 319 categorical detections with sharp differentiation: category-theory threads averaged 3 categorical patterns per thread vs. 0.2 for mathematical-physics threads. The signal is real.

How This Connects to First Proof 2

We did a proof sprint on the First Proof benchmark two weeks ago (10 research-level math problems, 55 hours, 4/10 correct). A detailed retrospective identified five things we'd do differently. The superpod run addresses three of them directly:

1. **Better literature mining.** During Sprint 1, keyword-based searches missed critical techniques (e.g., hyperbolic Hessian contraction bounds for Problem 4). The superpod output provides a structured index of how 667K threads discuss mathematical concepts — not just keywords, but typed connections between assumptions and conclusions. When we need a bridge between proof steps, we can query: "what threads have an output port matching this input?"
2. **Automated proof verification.** We built a verifier that checks each edge in a proof wiring

diagram for structural consistency (port types, discourse markers, categorical patterns). During Sprint 1 these checks were manual. Now they're automated, and the superpod gives the verifier a large corpus to calibrate against.

3. **Domain-depth pre-loading.** Sprint 1's two wrong answers were both on problems requiring deep specialized knowledge. The superpod output lets us pre-identify which subfields have rich coverage and which are sparse, so we can front-load targeted mining for the harder problems.

The other two lessons (mandatory falsification protocol, better calibration tracking) are process improvements that don't depend on the corpus but benefit from it — a verification score that plateaus is a concrete trigger for switching from constructive to falsification mode.

Timeline

- **This week:** Run the required superpod block (sharded, skip LLM). This produces CT-backed wiring, expression surfaces, hypergraphs, LWGM embeddings, and a FAISS similarity index.
- **Next 2 weeks:** Optional: backfill LLM stages (pattern tags + reverse morphogenesis) on a bounded thread sample for quality assessment. Integrate the verifier + structural search into the proof workflow.
- **Week 4:** First Proof Batch 2, if announced. Otherwise, dry-run the protocol on a practice problem (e.g., re-attempt Problem 4 targeting the all-n bridge we missed the first time).

What I'd Like From You

Mainly: a sanity check. Does this pipeline make sense as a way to build structured mathematical knowledge at scale? The bet is that typed wiring diagrams (not just embeddings, not just keyword search) are the right intermediate representation — rich enough to support automated verification, structured enough to enable port-level queries, but not so formal that we need full theorem-proving infrastructure.

If you want to poke at any of the artifacts, the repo is futon6 on GitHub (private, happy to add you). Key files:

```
scripts/superpod-job.py          # the pipeline (4,358 lines)
scripts/assemble-wiring.py       # wiring assembly (801 lines)
scripts/ct-verifier.py           # proof verifier (617 lines)
data/nlab-ct-reference.json      # CT reference (20K pages)
data/first-proof/superpod-handoff-rob.lit.md  # executable ledger
```